BA305 — Final Exam · Version 2 · Solutions

Total: 65 points. Part A: 40 points · Part B: 25 points.

Part A — Concepts (Questions 1–15, 40 points)

No code in this part.

Part A: 40 points.

Question 1 — True / False (2 points)

Naïve Bayes can only be used to classify text, such as emails.

- True
- False

> **Answer — False.** Naïve Bayes works with any categorical features. The course used it for spam (words) and for tax fraud (categories such as income bracket and refund claimed).

Question 2 — Multiple choice (3 points)

In spam emails, the words "free" and "money" generally appear together. How does a Naïve Bayes spam filter treat a message that contains both?

- (a) As two separate clues, so the evidence for spam is counted twice.
- (b) As one combined clue, because the model notices that the two words appear together.
- (c) It ignores both words, because words that generally appear together carry no information.
- (d) It cannot score the message, because Naïve Bayes needs the words to be independent.

> **Answer — (a).** Naïve Bayes treats the words as independent within each class, so it multiplies one factor for "free" and another for "money", even though they carry much the same evidence. That double counting is what "naïve" means. (b) The model never looks at pairs of words. (c) Each word still counts. (d) The assumption is wrong here, but the model still gives a score.

Question 3 — Multiple choice (3 points)

A spammer adds many everyday words, such as "meeting", "lunch" and "thanks", to an obvious spam message. The filter, a Naïve Bayes model trained on word counts, now labels the message as normal. Why?

- (a) Each added word is more common in normal messages and counts on its own, so with enough of them the normal score ends up larger than the spam score.

- (b) The filter reads only the first few words of a message, and the spam words come later.
- (c) The filter labels every long message as normal, whatever its words.
- (d) The filter ignores words that appeared in both spam and normal training messages.

> **Answer — (a).** Every word adds its own factor, P(word | class). Words like "lunch" are much more common in normal messages, so each one tilts the comparison toward normal; with enough of them the normal score wins. (b) Every word counts. (c) Length alone does not decide the label. (d) A word seen in both classes still counts, favoring the class where it is more common.

Question 4 — Select all that apply (4 points)

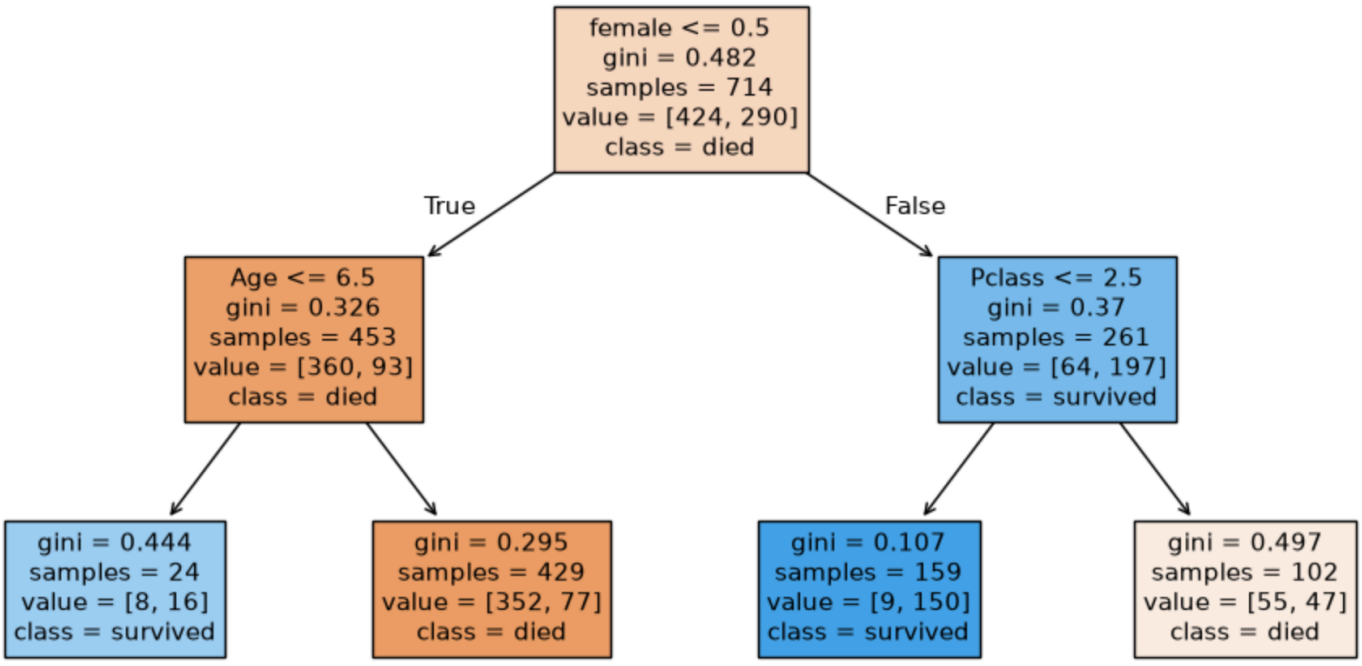
Select **all** statements about classification trees that are true.

- (a) A deeper tree usually fits the training data better than a shallower one.
- (b) A tree grown until every leaf is pure usually scores much lower on test data than on training data.
- (c) Limiting the depth always raises the tree's test accuracy.
- (d) A deeper tree is easier to explain to a manager than a shallower one.
- (e) Requiring a minimum number of records in each leaf also helps keep a tree from overfitting.

> **Answer — (a), (b) and (e).** (a) More splits let a deeper tree follow the training data more closely. (b) A fully grown tree memorizes the training data, noise included. (e) A minimum leaf size stops leaves built for a handful of records. (c) A tree that is too shallow underfits; "always" is false. (d) More levels mean more rules, so a deeper tree is harder to explain.

Question 5 — Select all that apply (3 points)

A tree was fit on 714 Titanic passengers to predict whether each passenger survived, using sex (`female`: 1 for a woman, 0 for a man), age in years (`Age`) and ticket class (`Pclass`: 1, 2 or 3).



Select **all** statements this tree supports.

- (a) A 5-year-old boy is predicted to survive.
- (b) Among women, those in first or second class survived more often than those in third class.
- (c) The tree uses ticket class to classify every passenger.
- (d) A 30-year-old woman in third class is predicted to survive.
- (e) Most men older than 6.5 years did not survive.

> **Answer — (a), (b), (e).** (a) A boy has `female` = 0 → left; age 5 ≤ 6.5 → the leaf [8, 16], survived. (b) Women in first or second class: 150 of 159 survived (94%); third class: 47 of 102 (46%). (e) Men older than 6.5: 352 of 429 died. (c) Men are split by age only. (d) A third-class woman lands in [55, 47]: died is the majority.

Question 6 — Multiple choice (2 points)

How does a classification tree choose each split?

- (a) It picks the split whose groups are the purest overall.
- (b) It picks the feature with the largest values.
- (c) It uses the features in the order of the columns in the data.
- (d) It picks a feature at random.

> **Answer — (a).** At each node the tree tries the possible splits and keeps the one whose groups are purest overall (for example, the lowest weighted Gini). (b) The size of the values plays no part. (c) Column order does not matter. (d) A single tree is not random.

Question 7 — Multiple choice (2 points)

The outcome that a decision tree predicts can be:

- (a) categorical only
- (b) numerical only
- (c) categorical or numerical
- (d) only a yes/no outcome

> **Answer — (c).** A classification tree predicts a category (the leaf's majority class); a regression tree predicts a number (the leaf's average).

Question 8 — Multiple choice (3 points)

A logistic regression gives a customer odds of 0.5 of buying. Which statement is right?

- (a) The customer is less likely to buy than not; the probability of buying is 0.33.
- (b) The customer is less likely to buy than not; the probability of buying is 0.5.
- (c) The customer is equally likely to buy or not.
- (d) The customer is more likely to buy than not; the probability of buying is 0.67.

> **Answer — (a).** Odds below 1 mean buying is less likely than not. p = 0.5 / 1.5 = 0.33: 1 buyer for every 2 non-buyers. (b) reads the odds as a probability. (c) would need odds of 1. (d) 0.67 is the probability of not buying.

Question 9 — True / False (3 points)

If a customer's odds of buying are larger than 0, then the customer's odds of not buying are less than 0.

- True
- False

> **Answer — False.** Odds = p / (1 − p) are never negative. The odds of not buying are 1 divided by the odds of buying (odds of 4 to buy mean odds of 1/4 not to buy), so they are positive too.

Question 10 — Multiple choice (2 points)

Which statement about logistic regression is true?

- (a) For each record, it outputs a probability between 0 and 1.
- (b) It is not suitable for classification tasks.
- (c) It requires every feature to follow a normal distribution.
- (d) It can only use features coded as 0 or 1.

> **Answer — (a).** The logistic function turns the linear score into a probability between 0 and 1. (b) Classification is what it is for. (c) It assumes nothing about how the features are distributed. (d) Numeric features are fine; only categories need dummies.

Question 11 — Multiple choice (3 points)

A hidden node receives two inputs, 1 and 0.5, with weights 0.4 and −0.8, respectively. It uses the logistic activation function. What does the node output?

- (a) 0.5
- (b) 0
- (c) 0.6
- (d) 1

> **Answer — (a).** Weighted sum: $0.4 \times 1 + (-0.8) \times 0.5 = 0$. Logistic: $1 / (1 + e^{0}) = 0.5$. (b) is the sum before the activation. (c) leaves out the second input. (d) is what a step function would give.

Question 12 — True / False (2 points)

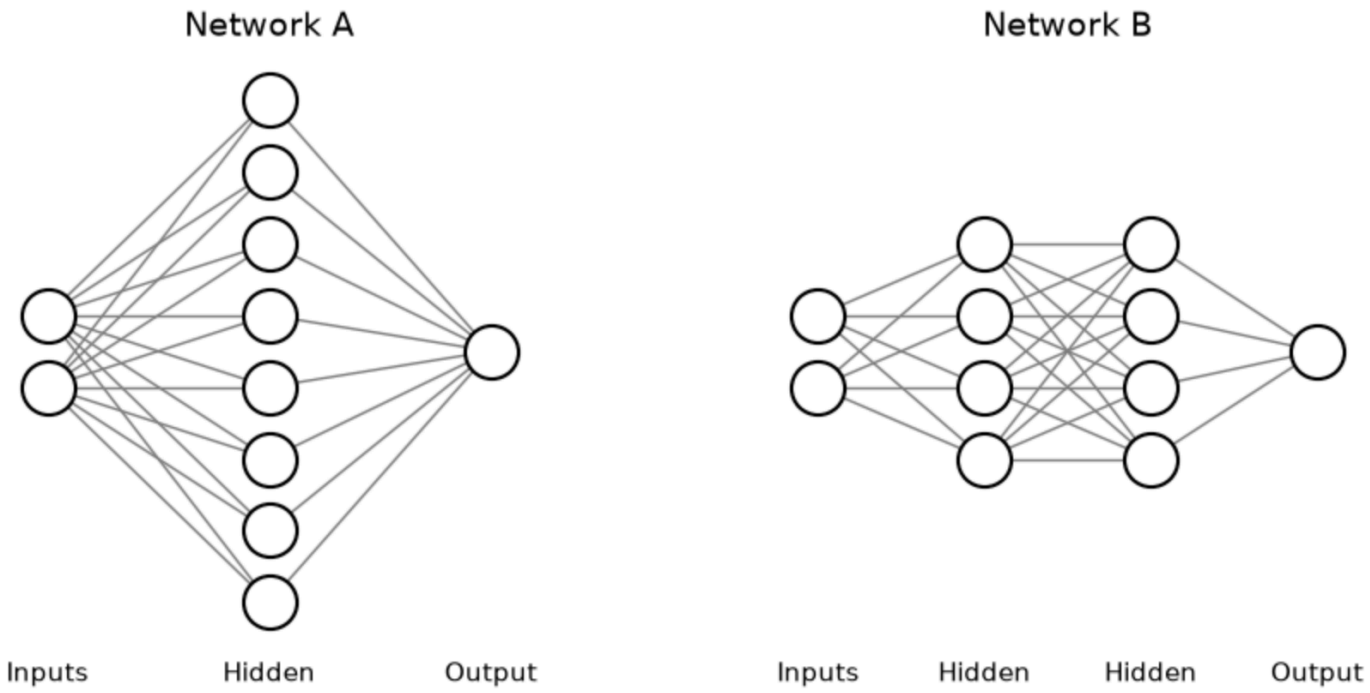
The same neural network architecture, for example three hidden layers of 5 nodes each (`hidden_layer_sizes=(5, 5, 5)`), can be used with `MLPRegressor` to predict a used car's price, or with `MLPClassifier` to predict whether a customer buys.

- True
- False

> **Answer — True.** The hidden layers are the same; only the output changes: one number for `MLPRegressor`, a probability for each class for `MLPClassifier`.

Question 13 — Multiple choice (3 points)

The two networks below have the same inputs, the same output and the same number of hidden nodes, spread over the hidden layers differently. Every node in one layer connects to every node in the next.



Both are trained on the same data. Which network is more likely to overfit?

- (a) Network A
- (b) Network B
- (c) Both are equally likely, because they have the same number of nodes
- (d) Neither, because overfitting depends only on the data, not on the network

> **Answer — (b).** More parameters (weights) give a network more room to memorize the training data. Count the connections: A has $2 \times 8 + 8 \times 1 = 24$; B has $2 \times 4 + 4 \times 4 + 4 \times 1 = 28$. (a) A wider layer does not always mean more parameters. (c) Parameters sit on the connections, not the nodes. (d) The model matters as well as the data.

Question 14 — Multiple choice (3 points)

An analyst scores a tree with 5-fold cross-validation. Which statement is true?

- (a) Every record is used for testing exactly once and for training four times.
- (b) The tree is trained once, on 80% of the data, and tested five times.
- (c) In each round, the tree is trained on one fold and tested on the other four.
- (d) The folds are drawn again in each round, so a record may be tested more than once.

> **Answer — (a).** The data is cut once into 5 folds. Each round tests on one fold and trains on the other four, so each record is tested once and used for training four times. (b) The model is trained 5 times. (c) It is the other way round. (d) The folds are fixed.

Question 15 — True / False (2 points)

Logistic regression is a good choice to predict how many dollars a customer will spend next month.

- True
- False

> **Answer — False.** Logistic regression predicts the probability of a class, not an amount. A dollar amount needs a regression model: linear regression, a regression tree, k-NN regression, or a neural network for regression.

Part B — Coding questions (Questions 16–23, 25 points)

The code and its output are in two separate handouts, **Handout B1** and **Handout B2**. Each question names the exhibits it uses. You cannot run the code.

Part B: 25 points. Total: 65 points.

Case B1 — Wildlife Strikes (use Handout B1)

US airlines report every collision between an aircraft and wildlife, mostly birds. This data covers strikes from 1990 to 2018 involving four large US airlines. It is in a DataFrame named `strikes`, one row per strike:

Column	Meaning
`operator`	Airline
`stage`	Phase of flight: Approach, Climb, Take-Off Run, Landing Roll or Other
`time_of_day`	Dawn, Day, Dusk or Night
`incident_month`	Month of the strike (1–12)
`height`	Height of the aircraft (feet)
`speed`	Speed of the aircraft (knots)
`repair_cost`	Cost of the repairs, in today's dollars. Airlines report a cost for only a few strikes, so 0 means no cost was reported, not that nothing was repaired
`damaged`	1 if the aircraft was damaged, 0 if not

An airline wants to predict, from the circumstances of a strike, whether the aircraft was damaged, to decide which aircraft to inspect first.

Exhibit 1

```
# Number of strikes, and the fraction that damaged the aircraft.
print('Strikes:', len(strikes))
strikes['damaged'].value_counts(normalize=True).round(3)
```

```
Strikes: 24798
damaged
0    0.929
1    0.071
Name: proportion, dtype: float64
```

Exhibit 2

```
# The first few strikes.
strikes.head()
```

	operator	stage	time_of_day	incident_month	height	speed	repair_cost	damaged
0	AMERICAN AIRLINES	Climb	Day	12	700.0	200.0	0.0	1
1	AMERICAN AIRLINES	Approach	Night	12	600.0	145.0	0.0	0
2	AMERICAN AIRLINES	Take-Off Run	Day	12	0.0	130.0	0.0	0
3	AMERICAN AIRLINES	Approach	Day	12	500.0	160.0	0.0	0
4	AMERICAN AIRLINES	Approach	Day	12	100.0	150.0	0.0	0

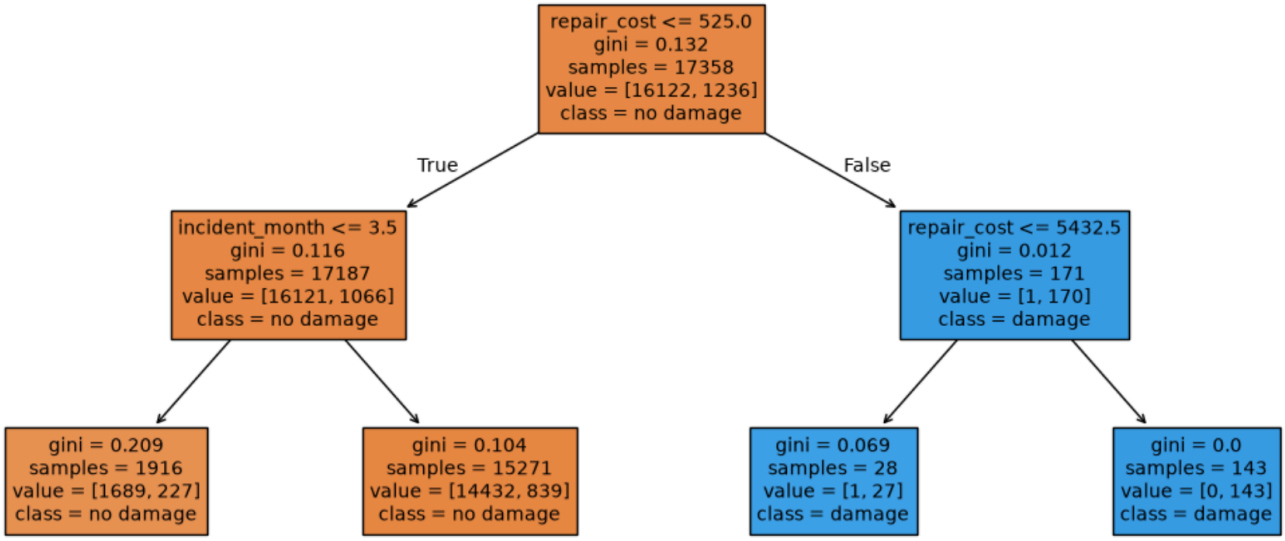
Question 16 — True / False (2 points)

Exhibit 3

```
# Take a first subset of the features. Hold out 30% for testing.
X = pd.get_dummies(strikes[['operator', 'stage', 'time_of_day', 'incident_month', 'repair_cost']],
dtype=int)
y = strikes['damaged']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)

tree_1 = DecisionTreeClassifier(max_depth=2, random_state=0).fit(X_train, y_train)
pred = tree_1.predict(X_test)
print('Precision:', round(precision_score(y_test, pred), 3))
plt.figure(figsize=(14, 6))
plot_tree(tree_1, feature_names=list(X.columns), class_names=['no damage', 'damage'], filled=True,
fontsize=10);
```

```
Precision: 0.986
```



The airline wants to decide which aircraft to inspect first, as soon as a strike is reported. A precision of 0.986 on strikes it never saw shows that the airline can use `tree_1`, the tree in Exhibit 3, to do that.

- True
- False

> **Answer — False.** The first split is on the repair cost, which exists only after the aircraft has been inspected and repaired, so it is unknown when the strike is reported. This is leakage, so the 0.986 precision cannot be reproduced when the prediction is needed.

Question 17 — Multiple choice (4 points)

Exhibit 4

```
# Fraction of each stage among the strikes without damage (0) and with damage (1).
table = pd.crosstab(strikes['stage'], strikes['damaged'], normalize='columns')
table.loc['Total'] = table.sum()
table.round(2)
```

damaged	0	1
stage		
Approach	0.53	0.45
Climb	0.20	0.31
Landing Roll	0.12	0.05
Other	0.02	0.03
Take-Off Run	0.14	0.16
Total	1.00	1.00

Exhibit 5

```
# Fraction of each time of day among the strikes without damage (0) and with damage (1).
table = pd.crosstab(strikes['time_of_day'], strikes['damaged'], normalize='columns')
table.loc['Total'] = table.sum()
table.round(2)
```

damaged	0	1
time_of_day		
Dawn	0.03	0.04
Day	0.59	0.59
Dusk	0.04	0.04
Night	0.33	0.33
Total	1.00	1.00

In both tables, column 0 holds the strikes without damage and column 1 the strikes with damage. For example, 0.45 in the `Approach` row of column 1 means that 45% of the damaging strikes happened on approach.

A Naïve Bayes model is trained on all the strikes in `strikes`, with `stage` and `time_of_day` as its only features. An aircraft is struck while climbing, at night. Which calculation and prediction are correct?

- (a) Damage: 0.071 × 0.31 × 0.33 = 0.00726; no damage: 0.929 × 0.20 × 0.33 = 0.06131. It predicts no damage.
- (b) Damage: 0.31 × 0.33 = 0.10230; no damage: 0.20 × 0.33 = 0.06600. It predicts damage.
- (c) Damage: 0.071 × 0.31 × 0.33 = 0.00726. This is far below 0.5, so it predicts no damage.
- (d) No calculation is needed: only 7.1% of strikes cause damage, so Naïve Bayes will always predict no damage.

> **Answer — (a).** Score = P(class) × P(climb | class) × P(night | class), with the class shares from Exhibit 1. Damage: 0.071 × 0.31 × 0.33 ≈ 0.0073. No damage: 0.929 × 0.20 × 0.33 ≈ 0.0613. The larger score wins: no damage, because damaging strikes are rare to begin with. (b) leaves out the class shares, which flips the answer. (c) The two scores are compared with each other, not with 0.5. (d) Strong evidence could outweigh the class share, so the calculation is needed.

Question 18 — Multiple choice (3 points)

Exhibit 6

```
# Take a new subset of the features, different from Exhibit 3. Hold out 30% for testing.
X = pd.get_dummies(strikes[['operator', 'stage', 'time_of_day', 'incident_month', 'height', 'speed']],
dtype=int)
y = strikes['damaged']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
print('Fraction of damaged aircraft in the test set:', round(y_test.mean(), 3))
```

```
Fraction of damaged aircraft in the test set: 0.071
```

Exhibit 7

```
# Trees of different depths: size, accuracy and recall.
rows = []
for depth in [1, 2, 4, 6, 10, None]:
    t = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(X_train, y_train)
    pred = t.predict(X_test)
    rows.append({'max_depth': depth if depth else 'no limit', 'leaves': t.get_n_leaves(),
                 'train accuracy': round(accuracy_score(y_train, t.predict(X_train)), 3),
                 'test accuracy': round(accuracy_score(y_test, pred), 3),
                 'recall': round(recall_score(y_test, pred), 3)})
pd.DataFrame(rows)
```

	max_depth	leaves	train accuracy	test accuracy	recall
0	1	2	0.929	0.929	0.000
1	2	4	0.929	0.929	0.000
2	4	16	0.929	0.929	0.000
3	6	61	0.931	0.927	0.021
4	10	392	0.938	0.922	0.057
5	no limit	2698	0.976	0.888	0.132

Which statement does Exhibit 7 support?

- (a) The trees of depth 1, 2 and 4 do no better than predicting "no damage" for every strike.
- (b) The tree with no limit is the best of the six, because it catches every damaging strike.
- (c) The depth-6 tree is a good model, because its training and test accuracies are both above 0.92.
- (d) The trees that catch more damaging strikes have higher test accuracy.

> **Answer — (a).** 7.1% of the test strikes are damaging, so "no damage for everyone" scores 0.929, exactly the test accuracy of depths 1–4, which catch no damaging strikes (recall 0). (b) No limit catches only 13.2% (recall 0.132) and overfits (train 0.976, test 0.888). (c) 0.927 is below the naïve 0.929, with recall 0.021. (d) As recall rises, test accuracy falls.

Question 19 — Short answer (4 points)

Exhibit 8

```
# Depth-6 tree, without and with class_weight='balanced' (the damaging strikes together then weigh as
much as the non-damaging ones).
for weight in [None, 'balanced']:
    t = DecisionTreeClassifier(max_depth=6, class_weight=weight, random_state=0).fit(X_train,
y_train)
    pred = t.predict(X_test)
    print('class_weight =', weight, '   test accuracy:', round(accuracy_score(y_test, pred), 3))
```

```
class_weight = None    test accuracy: 0.927
class_weight = balanced    test accuracy: 0.751
```

Exhibit 9

```
# Confusion matrix of the depth-6 tree without the weighting.
tree_6 = DecisionTreeClassifier(max_depth=6, random_state=0).fit(X_train, y_train)
ConfusionMatrixDisplay(confusion_matrix(y_test, tree_6.predict(X_test)), display_labels=['no damage',
'damage']).plot(colorbar=False)
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.title("class_weight = None");
```

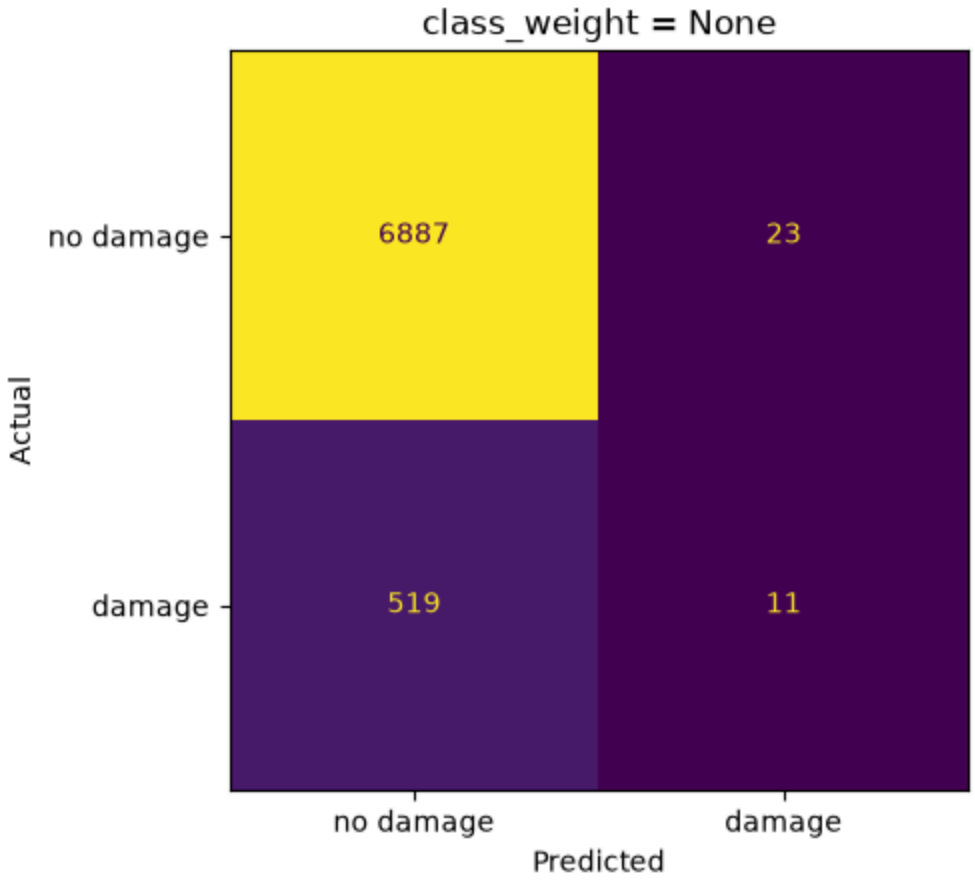
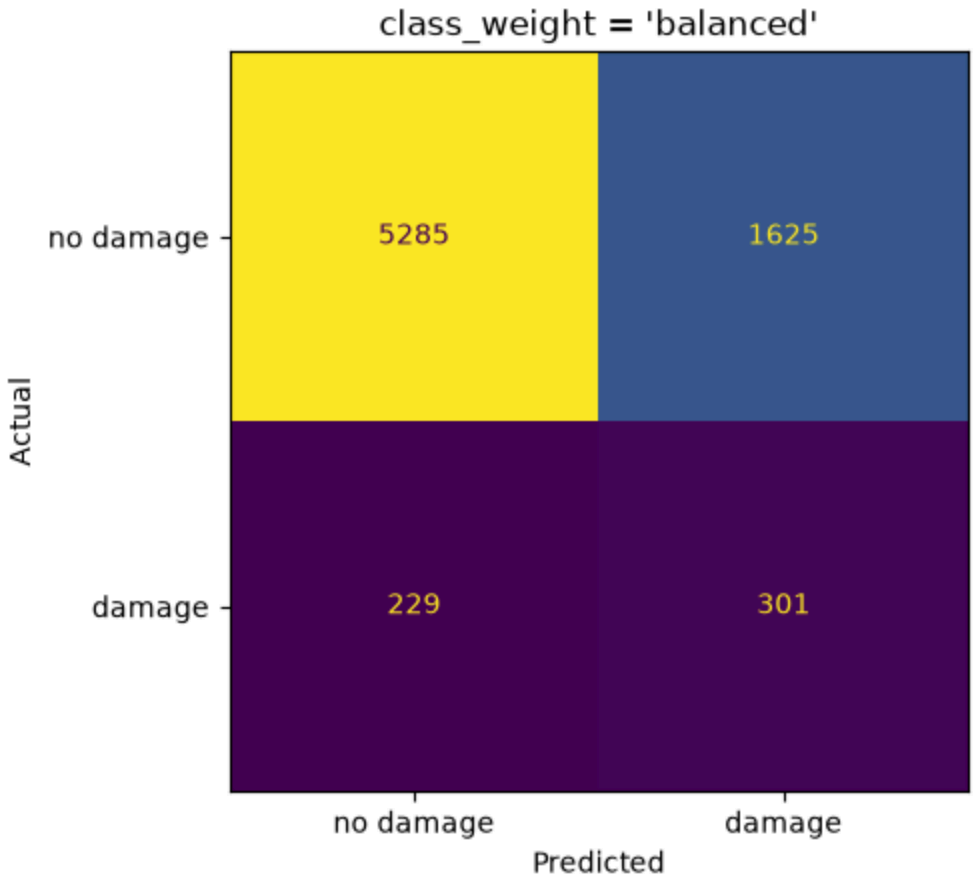


Exhibit 10

```
# Confusion matrix of the depth-6 tree with class_weight='balanced'.
tree_w = DecisionTreeClassifier(max_depth=6, class_weight='balanced', random_state=0).fit(X_train,
y_train)
ConfusionMatrixDisplay(confusion_matrix(y_test, tree_w.predict(X_test)), display_labels=['no damage',
'damage']).plot(colorbar=False)
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.title("class_weight = 'balanced'");
```



The airline uses a depth-6 tree to choose which aircraft to inspect. With `class_weight='balanced'`, the depth-6 tree's test accuracy fell from 0.927 to 0.751 (Exhibit 8). Using the confusion matrices in Exhibits 9 and 10, explain whether the weighted tree `tree_w` raises more false alarms (undamaged aircraft sent for inspection) or misses more damaged aircraft than the unweighted tree `tree_6`. The airline's priority is to inspect as many of the damaged aircraft as possible, even if that means inspecting some undamaged ones. Which tree should it use, and which metric did you use to pick the tree? You do not need to calculate the metric. Answer in at most 3 sentences.

> **Model answer.** The weighted tree raises far more false alarms (1,625 against 23) but misses far fewer damaged aircraft (229 against 519). Catching damaged aircraft is the priority, so the airline should use the weighted tree `tree_w`, chosen by recall for damaged aircraft (about 0.57 against 0.02; no calculation needed).

Case B2 — Orange Juice Purchases (use Handout B2)

A grocery chain recorded 1,070 purchases of orange juice in five of its stores. In each, the customer bought either Citrus Hill (CH) or Minute Maid (MM). The data is in a DataFrame named `juice`:

Column	Meaning
`MM`	1 if the customer bought Minute Maid, 0 if Citrus Hill
`WeekofPurchase`	Week of the purchase, as a running number (227 to 278); a higher number means a later week
`LoyalCH`	The customer's loyalty to Citrus Hill, from 0 to 1, based on past purchases
`PriceDiff`	Price paid for Minute Maid minus price paid for Citrus Hill ($)
`SpecialMM`	1 if Minute Maid was on special that week

Citrus Hill wants to predict which customers will buy Minute Maid, so it can send them a coupon.

Exhibit 1

```
# Number of purchases, and the fraction that were Minute Maid.
print('Purchases:', len(juice))
juice['MM'].value_counts(normalize=True).round(3)
```

```
Purchases: 1070
MM
0    0.61
1    0.39
Name: proportion, dtype: float64
```

Exhibit 2

```
# The first few purchases.
juice.head()
```

	MM	WeekofPurchase	LoyalCH	PriceDiff	SpecialMM
0	0	237	0.500000	0.24	0
1	0	239	0.600000	-0.06	1
2	0	245	0.680000	0.40	0
3	1	227	0.400000	0.00	0
4	0	228	0.956535	0.00	0

Exhibit 3

```
# Features: every column except the target. Hold out 30% for testing.
X = juice.drop(columns=['MM'])
y = juice['MM']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
```

Question 20 — Multiple choice (2 points)

Before sending coupons based on any model, Citrus Hill wants a fair test on customers the model has not seen. In Exhibit 3, the analyst splits the data with `stratify=y`. What does that do?

- (a) About 0.39 of the purchases in both the training set and the test set are Minute Maid.
- (b) About 0.3 of the purchases in both the training set and the test set are Minute Maid.
- (c) The training set and the test set each get 0.5 of the purchases.
- (d) The Citrus Hill purchases (0.61) go to the training set and the Minute Maid purchases (0.39) to the test set.

> **Answer — (a).** `stratify=y` keeps the target's mix the same in both parts: 0.39 of all purchases were Minute Maid (Exhibit 1), so about 0.39 of the training purchases and of the test purchases are. (b) 0.3 is `test_size`, the share held out for testing. (c) The split is 0.7 / 0.3. (d) Both sets keep both classes.

Question 21 — Multiple choice (3 points)

Exhibit 4

```
# Logistic regression with three features: loyalty to Citrus Hill, the price gap, and whether Minute
Maid is on special.
small = ['LoyalCH', 'PriceDiff', 'SpecialMM']
log_reg_small = LogisticRegression(max_iter=5000).fit(X_train[small], y_train)
pd.DataFrame({'coefficient': log_reg_small.coef_[0], 'exp(coefficient)':
np.exp(log_reg_small.coef_[0])},
             index=small).round(3)
```

	coefficient	exp(coefficient)
LoyalCH	-5.128	0.006
PriceDiff	-1.699	0.183
SpecialMM	0.236	1.266

Citrus Hill's marketing team wants to know what pushes customers toward Minute Maid, to decide when a coupon is worth sending. Holding the other features fixed, which statement does `log_reg_small`, the logistic regression in Exhibit 4, support?

- (a) A customer with a higher `LoyalCH` value is more likely to buy Minute Maid.
- (b) If Minute Maid's price rises by $1 compared with Citrus Hill's, the odds of buying Minute Maid are multiplied by 0.183.
- (c) Minute Maid being on special raises the probability of buying Minute Maid by 0.236.
- (d) A customer with `SpecialMM` = 1 is less likely to buy Minute Maid, because its coefficient is below 1.

> **Answer — (b).** exp(−1.699) = 0.183: each $1 more that Minute Maid costs compared with Citrus Hill multiplies the odds of buying it by 0.183. (a) `LoyalCH` has a negative coefficient (−5.128). (c) 0.236 is added to the log-odds, not to the probability. (d) `SpecialMM`'s coefficient is above 0 (exp 1.266, above 1), so a special makes Minute Maid more likely; the cutoff of 1 is for exp(coefficient).

Question 22 — Multiple choice (3 points)

Exhibit 5

```
# The same neural network, trained on the raw features and on features scaled to 0-1.
scaler = MinMaxScaler().fit(X_train)
inputs = {'raw': (X_train, X_test), 'scaled 0-1': (scaler.transform(X_train),
scaler.transform(X_test))}

rows = []
for name, (Xtr, Xte) in inputs.items():
    nn = MLPClassifier(hidden_layer_sizes=(5, 5, 5), activation='logistic', solver='lbfgs',
random_state=1).fit(Xtr, y_train)
    rows.append({'features': name, 'training steps': nn.n_iter_,
                 'train accuracy': round(accuracy_score(y_train, nn.predict(Xtr)), 3),
                 'test accuracy': round(accuracy_score(y_test, nn.predict(Xte)), 3),
                 'fraction predicted MM': round(nn.predict(Xte).mean(), 3)})
pd.DataFrame(rows)
```

	features	training steps	train accuracy	test accuracy	fraction predicted MM
0	raw	2	0.610	0.611	0.000
1	scaled 0-1	200	0.826	0.857	0.377

In Exhibit 5 Citrus Hill trains the same neural network twice: once on the raw features and once on the features scaled to 0–1. Which statement best describes the network trained on the raw features?

- (a) It stopped after 2 steps because it had already found the best fit for the training customers.
- (b) It overfit: it learned the training customers too closely, which is why its test accuracy is low.
- (c) It trained properly: raw features suit neural networks, and 0.611 is a reasonable score for this data.
- (d) It underfit: it learned almost nothing, so it does no better than predicting Citrus Hill for everyone, even on the training customers.

> **Answer — (d).** 2 training steps and 0 predicted MM: it labels everyone Citrus Hill, so its training (0.610) and test (0.611) accuracies just equal the Citrus Hill share (0.61). Raw features such as `WeekofPurchase` (in the hundreds) stall training; scaled to 0–1, the same network reaches 0.857. (a) It stopped because training stalled, not because it fit well. (b) Training accuracy is not higher than test accuracy, so it did not overfit. (c) 0.611 is the "always Citrus Hill" rule.

Question 23 — Short answer (4 points)

Exhibit 6

```
# Logistic regression (all four features, standardized) gives each test customer a probability of
buying Minute Maid.
std = StandardScaler().fit(X_train)
log_reg = LogisticRegression(max_iter=5000).fit(std.transform(X_train), y_train)
prob = log_reg.predict_proba(std.transform(X_test))[:, 1]

# Send a coupon when the probability is at or above the threshold.
rows = []
for t in [0.2, 0.3, 0.4, 0.5, 0.6, 0.7]:
    coupon = (prob >= t).astype(int)
    rows.append({'threshold': t, 'coupons sent': coupon.sum(),
                 'sent, bought CH anyway': ((coupon == 1) & (y_test == 0)).sum(),
                 'not sent, bought MM': ((coupon == 0) & (y_test == 1)).sum(),
                 'precision': round(precision_score(y_test, coupon), 3),
                 'recall': round(recall_score(y_test, coupon), 3)})
pd.DataFrame(rows)
```

	threshold	coupons sent	sent, bought CH anyway	not sent, bought MM	precision	recall
0	0.2	200	78	3	0.610	0.976
1	0.3	168	52	9	0.690	0.928
2	0.4	143	33	15	0.769	0.880
3	0.5	117	23	31	0.803	0.752
4	0.6	97	12	40	0.876	0.680
5	0.7	76	8	57	0.895	0.544

Citrus Hill will send a coupon to every customer whose probability of buying Minute Maid, as predicted by `log_reg` in Exhibit 6, is at or above a threshold. A coupon sent to a customer who would buy Citrus Hill anyway wastes $1. A Minute Maid buyer who gets no coupon costs Citrus Hill $2 in lost profit. Which threshold gives Citrus Hill the lowest total cost on these test customers? Give that total cost, and the percentage of the Minute Maid buyers who get no coupon at that threshold. Answer in at most 3 sentences.

> **Model answer — 0.4.** Cost = 1 × (sent, bought CH anyway) + 2 × (not sent, bought MM): 0.2 → $84, 0.3 → $70, 0.4 → 33 + 2 × 15 = $63, 0.5 → $85, 0.6 → $92, 0.7 → $122. At 0.4 the recall is 0.880, so 12.0% of the Minute Maid buyers get no coupon (15 of 125).